Title

- **TripEase**
- Travel package browsing and booking system
- Developer: Jaycel Aplacador
- Client: TripEase project stakeholders

Client and Problem

- Travelers need a clear way to find packages, make bookings, and track progress.
- Administrators need a central workflow for packages, bookings, payments, and current totals.
- TripEase connects these needs through a React application and a Laravel API.

Solution and Roles

- **Customer:** browses and searches packages, books trips, manages pending bookings, and views personal booking history.
- **Admin:** manages packages, reviews bookings, changes booking status, manages payments, and views dashboard totals.
- Role checks and customer ownership rules protect privileged actions and booking data.

Features

- Sanctum-based registration, login, and API authentication.
- Searchable, filterable, paginated travel packages.
- Booking creation with server-calculated totals and availability handling.
- Customer-scoped bookings and cancellation of eligible pending bookings.
- Admin booking workflow: Pending -> Approved -> Completed, or Pending -> Cancelled.
- Payment management and database-backed dashboard counts.
- Responsive React interface with form and request feedback.

Architecture and ERD

- React 19, React Router, Axios, and Vite frontend.
- Laravel 13 JSON API, Sanctum authentication, role middleware, validation, and API Resources.
- MySQL for application data; automated backend tests use in-memory SQLite.
- Main relationships: User -> Bookings; Travel Package -> Bookings; Booking -> optional Payment.
- Show `ERD.mmd` and the React -> Axios -> Laravel API -> database request path.

Live Demo Flow

1. Sign in as a customer and search/filter the package list.
2. Open a package and create a booking; show its Pending status and customer booking list.
3. Sign in as an admin and review the new booking and dashboard totals.
4. Approve the booking, then show the customer-visible status.
5. Complete the booking and show the terminal status.
6. Demonstrate the alternative Pending -> Cancelled path, if time permits.

Handover and Security

- Handover documents: `README.md`, `ERD.mmd`, and `postman_collection.json`.
- Configure `.env` for the target database; generate an application key and run migrations/seeders as documented.
- Seeded demo credentials are for local demonstration only; configure a secure demo password and never use fallback credentials in production.
- Protected API routes require Sanctum tokens; admin operations also require the admin role.
- UI screenshots, an exported ERD image, and a live Postman walkthrough remain manual handover items.

Closing

- **Delivered:** customer booking workflow, admin management workflows, API access controls, dashboard, documentation, and automated backend coverage.
- **Limitations:** full browser-to-database verification and production deployment/security review have not been completed; some UI states still require manual verification.
- **Phase 2 ideas (subject to approval):** payment-gateway integration, booking notifications, and expanded reporting.
- Thank you.